

AUTOESTUDO · BOOTCAMP QI TECH 2026

Autoestudo da Aula 2

Modelagem

Os **15 conceitos** da Aula 2, em três blocos — rotas e contratos, banco de dados e fluxos. Para cada conceito, os materiais escolhidos, com **tipo, idioma e tamanho** declarados, e a razão de cada um estar aqui. No fim, um **caminho mínimo de ~2h17** para quem só tiver três horas nesta semana.

Como ler esta lista

#1 #2 #3

DOC OFICIAL

ARTIGO

DOC + LABS

EM PORTUGUÊS

EM INGLÊS

~10–15 min

barra azul

A ordem em que a curadoria apresentou os materiais de cada conceito.

Vem de quem mantém a coisa: MDN, OWASP, AWS, Stripe, GitHub, Microsoft.

Texto de autor ou de veículo — inclui **blog, artigo técnico, post original** e **discussão**.

Tem prática junto, não só leitura — inclui o **curso em vídeo**.

O idioma do material. **Metade da lista está em português**; onde só havia opção boa em inglês, está dito.

O tamanho ou a duração declarados — para você escolher pelo tempo que tem.

A escolha default de cada conceito. Se for ler um só, é esse. O que estiver em bloco cinza são os outros materiais do mesmo conceito — complemento, contraponto ou próximo passo, conforme a nota de cada um.

BLOCO 1 — ROTAS E CONTRATOS

1+2 Cliente/servidor · HTTP (métodos, semântica, status codes)

Agrupados: o mesmo material da MDN amarra os dois conceitos num só fio — ciclo requisição/resposta, depois verbo, depois código de resposta.

#1

DOC OFICIAL

EM PORTUGUÊS

~10–15 min

Visão geral do HTTP — MDN Web Docs

Escolha default. Único material que fecha cliente/servidor + ciclo de conexão + estrutura de requisição/resposta num só lugar, com diagrama, sem pressupor nada. Tradução oficial da Mozilla, sempre gratuita. Não aprofunda semântica de status code — para isso, ver #3.

developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Overview

OS OUTROS MATERIAIS DESTE CONCEITO

#2

DOC OFICIAL

EM PORTUGUÊS

~5–8 min

Métodos de Requisição HTTP — MDN Web Docs

Referência objetiva de cada verbo (GET/POST/PUT/PATCH/DELETE) já com a semântica de idempotência e segurança embutida — exatamente o que “modelagem de rotas” cobra. Não cobre status codes.

developer.mozilla.org/pt-BR/docs/Web/HTTP/Methods

#3 **DOC OFICIAL** **EM PORTUGUÊS** leitura longa (referência) **Referência de Status HTTP — MDN Web Docs**

Organiza por faixa (2xx/3xx/4xx/5xx) e já traz, na entrada do 404, a frase-chave para a Aula 2: servidores podem devolver 404 no lugar de 403 **para esconder a existência de um recurso de quem não tem acesso** — o argumento exato por trás da decisão R8 (404, não 403, na transação).

developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Status

3 REST

#1 **ARTIGO** **EM INGLÊS** ~2.000 palavras (~8 min) **What is REST? REST API Definition for Beginners — freeCodeCamp (2022)**

Escolha default. Analogia de restaurante (API = garçom entre cliente e cozinha) torna REST tangível para quem nunca viu uma API. Não cobre as 6 constraints formais da arquitetura — para isso, #2.

www.freecodecamp.org/news/what-is-rest-rest-api-definition-for-beginners/

OS OUTROS MATERIAIS DESTE CONCEITO

#2 **ARTIGO** **EM INGLÊS** ~5–7 min **O que é uma API REST (API RESTful)? — IBM**

Cobre as **6 constraints arquiteturais** de REST (stateless, cacheable, layered system, interface uniforme, client-server, code-on-demand) — o que faz REST ser *arquitetura com restrições*, não “API com JSON e verbos”. Importa aqui porque o programa pede conceito, não receita.

www.ibm.com/think/topics/rest-apis

| Idioma: inglês (verificado nesta pesquisa, apesar do título indexado em PT).

#3 **ARTIGO** **EM PORTUGUÊS** ~9 min **O que é API REST? — Engineering Brasil (atualizado dez/2025)**

Única opção em português verificada nesta pesquisa; foca em benefícios/casos de uso (visão de produto) mais que nas constraints formais — bom complemento, não substitui o #2.

blog.engdb.com.br/o-que-e-rest-api/

4 Modelagem de rotas: contrato, schema × DTO, paginação, query-params

#1 **DOC OFICIAL** **EM INGLÊS** ~2.000 palavras **Using pagination in the REST API — GitHub Docs**

Escolha default. É contrato real, em produção, de uma API que todo aluno já tocou (a do GitHub, para clonar/subir código). Mostra page/per_page, cursor (before/after), Link header, com exemplo curl — o “como” que a aula teórica não vai dar. Só cobre paginação e query-params, não contrato/DTO em geral.

docs.github.com/en/rest/using-the-rest-api/using-pagination-in-the-rest-api

OS OUTROS MATERIAIS DESTE CONCEITO

#2 **ARTIGO** **EM INGLÊS** ~3.500–4.000 palavras **API Contract: What It Is and How to Write One — Moesif (mai/2026)**

Fecha o resto do conceito: o que entra num contrato (endpoints, formatos de request/response, autenticação, catálogo de erro, versionamento, SLA), com exemplos atuais (Idempotency-Key, headers Sunset/Deprecation). Assume familiaridade básica com o vocabulário de API.

www.moesif.com/blog/technical/api-contract/

#3 **ARTIGO** **EM INGLÊS** ~2.500 palavras **GraphQL Schemas vs. RESTful DTOs — Khalil Stemmler**

Material mais direto sobre schema × DTO como camadas distintas. Pressupõe DDD/CQRS e usa GraphQL como contraponto — leia focado na metade sobre REST/DTO, ignore o resto.

khalilstemmler.com/articles/graphql/graphql-schemas-vs-restful-dtos/

PARA IR ALÉM — NÃO É PISO

REFERÊNCIA **EM INGLÊS** **Zalando RESTful API Guidelines**

São a referência mais completa do mercado nessa faixa — 150+ regras cobrindo paginação, erros, versionamento, hypermedia. Verificado nesta pesquisa: pressupõem familiaridade prévia com REST/HTTP (“intermediário-avançado”). Não entram no piso da turma, mas valem como próximo passo para quem passar rápido pelos 3 acima.

opensource.zalando.com/restful-api-guidelines/

5 IDOR

#1 ARTIGO EM PORTUGUÊS ~2.500 palavras O que é IDOR e por que é uma vulnerabilidade tão comum? —

BugHunt

Escolha default. Plataforma brasileira de bug bounty; explica a causa raiz que interessa aqui — times validam *autenticação* ("quem é você") e esquecem *autorização* ("essa transação é sua?"), que é exatamente a distinção que a R8 do desafio cobra.

blog.bughunt.com.br/idor-vulnerabilidade-comum/

OS OUTROS MATERIAIS DESTE CONCEITO

#2 DOC OFICIAL EM INGLÊS ~850 palavras (~4 min) Insecure Direct Object Reference (IDOR) — OWASP

Fonte primária do termo — curta, direta, com exemplo em código Python. Complementa o #1 com a autoridade da OWASP.

owasp.org/www-community/attacks/insecure_direct_object_reference

#3 DOC + LABS EM INGLÊS Insecure direct object references (IDOR) — PortSwigger Web Security Academy

Único com **labs gratuitos** para explorar IDOR de verdade (não só ler sobre). O texto sozinho pressupõe alguma familiaridade com URLs e parâmetros — ainda assim, acessível.

portswigger.net/web-security/access-control/idor

BLOCO 2 — BANCO DE DADOS

6 Relacional e ACID

#1 ARTIGO EM PORTUGUÊS ~8–10 min Bancos de dados ACID — atômidade, consistência, isolamento e durabilidade explicados — freeCodeCamp Portuguese (jul/2024)

Escolha default. As 4 propriedades com analogias de mundo real (restaurante self-service, biblioteca, dois clientes disputando o último hambúrguer) — nenhum SQL prévio exigido, exatamente o piso da turma.

www.freecodecamp.org/portuguese/news/bancos-de-dados-acid-atomicidade-consistencia-isolamento-e-durabilidade-explicados/

OS OUTROS MATERIAIS DESTE CONCEITO

#2 DOC OFICIAL EM INGLÊS ~3.500 palavras ACID vs BASE Databases — AWS

Usa como exemplo central **transferência bancária** — o próprio domínio do desafio — e amarra ACID ao teorema CAP (bônus conceitual, não obrigatório). Boa segunda leitura para fixar antes de ver o gabarito.

aws.amazon.com/compare/the-difference-between-acid-and-base-database/

#3 CURSO EM VÍDEO EM PORTUGUÊS gratuito MySQL — 40 Horas — Curso em Vídeo (Gustavo Guanabara)

É o material que a turma já conhece, no canal do Slack do evento. Cobre o **modelo relacional na prática** — tabelas, chaves estrangeiras, JOIN — que os dois acima não ensinam. **Verificado nesta pesquisa: não cobre ACID, transação nem chave primária como tópico dedicado** — soma-se ao #1, não o substitui.

www.cursoemvideo.com/curso/mysql/

7 UUID e chave pública ≠ id do banco

#1 ARTIGO EM PORTUGUÊS ~2.500 palavras Pretende usar UUID como PK em vez de Int/BigInt? —

rponte.com.br (2021)

Escolha default. Propõe exatamente o modelo híbrido — id interno (Int/BigInt) para PK/FK + `external_id` (UUID) para exposição — que é a R5 do desafio ("nenhum identificador devolvido é o número que o banco gerou").

rponte.com.br/2021/01/30/nao-use-uuid-como-pk-nas-tabelas-do-seu-banco-de-dados/

OS OUTROS MATERIAIS DESTE CONCEITO

#2 ARTIGO EM INGLÊS moderado **UUID vs Auto Increment: How to Choose — Bytebase (2024)**

Mais atual: cobre o **UUIDv7** (ordenável por tempo, padronizado pela IETF em 2024) — o estado da arte que o artigo de 2021 não tinha como citar.

www.bytebase.com/blog/choose-primary-key-uuid-or-auto-increment/

#3 ARTIGO / DISCUSSÃO EM PORTUGUÊS **Quando utilizar UUID e quando utilizar auto-increment? — TabNews**

Contraponto saudável ao #1: visão mais cética — “use incremento até provar que UUID é melhor” — e desmonta o mito de que UUID por si só é uma medida de segurança.

www.tabnews.com.br/GabriPires/quando-utilizar-uuid-e-quanto-utilizar-auto-increment

8 Append-only (histórico imutável; por que soft delete não é append-only)

#1 ARTIGO EM INGLÊS ~1.800 palavras **Avoiding the soft delete anti-pattern — Cultured Systems (2024)**

Escolha default. Explica com precisão por que soft delete “engana” o banco (quebra UNIQUE, quebra FK) e por que isso é diferente de nunca apagar de verdade — o argumento por trás da R4 (“nada que entrou deixa de existir”).

www.cultured.systems/2024/04/24/Soft-delete/

OS OUTROS MATERIAIS DESTE CONCEITO

#2 ARTIGO EM INGLÊS moderado **Soft delete vs hard delete — AppMaster (2026)**

Contraponto equilibrado ao #1: quando soft delete é apropriado (LGPD/GDPR, dados sensíveis) — útil para o aluno não generalizar “soft delete é sempre errado”.

NÃO ABRIU NA VERIFICAÇÃO DE 13/09/2026 — HTTP 502

appmaster.io/blog/soft-delete-vs-hard-delete

O endereço está aqui porque o item é do material original, mas **não virou link clicável**: a página devolveu HTTP 502 em 5 tentativas nesta data (a raiz do site responde 200). Regra desta edição: link que não foi medido com 200 não compila.

#3 REFERÊNCIA EM INGLÊS ~3.500 palavras **Append-only — Wikipédia**

Mostra que append-only é conceito de sistemas mais amplo (log-structured storage, blockchains, filesystems) — ajuda a enxergar que a decisão de modelagem do desafio é uma instância de um padrão bem mais geral.

en.wikipedia.org/wiki/Append-only

9 Enumeradores; o que vira status x o que vira evento

#1 DOC / EBOOK EM INGLÊS ~4.000 palavras **Domain events: Design and implementation — Microsoft Learn**

Escolha default, com ressalva. É o material mais claro sobre a distinção “status é propriedade simples; evento é fato imutável que outras partes do sistema reagem” — o cerne do conceito 9. Usa jargão de DDD/C# (agregados, MediatR) que a turma não tem; leia pelo conceito, ignore o código.

learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/domain-events-design-implementation

OS OUTROS MATERIAIS DESTE CONCEITO

#2 BLOG EM INGLÊS ~7 min **How To Use Domain Events To Build Loosely Coupled Systems — Milan Jovanović**

Reforça o mesmo ponto de forma mais curta e com exemplo único (CourseCompletedDomainEvent) — bom para quem achar o #1 longo demais.

milanjovanovic.tech/blog/how-to-use-domain-events-to-build-loosely-coupled-systems

Sem 3º item. A metade “enumeradores” deste conceito (quando um ENUM de banco basta e quando vale uma tabela de domínio à parte) não teve fonte gratuita, atual e acessível confirmada nesta pesquisa.

10 Representação de dinheiro (centavos x decimal x float)

#1 ARTIGO EM PORTUGUÊS ~2.500 palavras Pare de usar FLOAT pra dinheiro (antes que dê ruim no seu sistema) — Programadores Deprê (mai/2025)

Escolha default. Mostra o erro de arredondamento com exemplo direto ($0.1 + 0.2$), explica por que ele se acumula em sistemas financeiros, e apresenta as duas saídas certas: decimal/bcadd e centavos inteiros.

programadoresdepre.com.br/pare-de-usar-float-pra-dinheiro-antes-que-de-ruim-no-seu-sistema/

OS OUTROS MATERIAIS DESTE CONCEITO

#2 DOC OFICIAL EM PORTUGUÊS referência Moedas aceitas — Stripe (docs, versão PT-BR)

Mostra a prática real em produção: a Stripe cobra sempre na **menor unidade da moeda** (centavos inteiros) — prova de que isso não é purismo acadêmico, é como sistemas de dinheiro de verdade são feitos. Não explica o *porquê* matemático do float — para isso, #1 ou #3.

docs.stripe.com/currencies

#3 ARTIGO EM INGLÊS ~1.500 palavras Never Use Floats for Money — husobee (2016)

Mais antigo (2016), mas a matemática de ponto flutuante não mudou: exemplos numéricos diretos ($165 * 1.40 = 230.99999999999997$) tornam o erro impossível de não notar.

husobee.github.io/money/float/2016/09/23/never-use-floats-for-currency.html

BLOCO 3 — FLUXOS

11 Máquina de estados

#1 ARTIGO EM PORTUGUÊS ~3.000–4.000 palavras (com código) State — Refactoring Guru

Escolha default. Único material em PT-BR verificado; parte do problema real (if/switch empilhados para tratar estado) até a solução, com exemplos em 10 linguagens. Visualmente muito bom, o que ajuda quem nunca pensou em código como estados.

refactoring.guru/pt-br/design-patterns/state

OS OUTROS MATERIAIS DESTE CONCEITO

#2 ARTIGO EM INGLÊS ~1.000–1.500 palavras Finite State Machine Explained — freeCodeCamp (2020)

Mais antigo (2020), mas o conceito (estados, transições, DFA/NFA) não muda com o tempo. Exemplo de máquina de café é mais simples que o do #1.

www.freecodecamp.org/news/finite-state-machines/

#3 DOC OFICIAL EM INGLÊS ~2.500–3.000 palavras (~8–10 min) State machines and statecharts — Stately (docs do XState)

Atual e muito acessível; vai além do básico (estados paralelos, hierárquicos) sem exigir a biblioteca XState para entender o conceito. Bom “próximo passo” depois do #1/#2.

stately.ai/docs/state-machines-and-statecharts

12 Concorrência e locks (pessimista, otimista, deixar o banco decidir)

#1 ARTIGO EM INGLÊS ~1.200–1.500 palavras Optimistic vs. Pessimistic Locking — Vlad Mihalcea

Escolha default. Referência de peso no tema (performance de JPA/Hibernate); explica os dois modelos com o cenário clássico de “lost update” (Alice/Bob disputando o mesmo saldo) — o cenário exato do desafio.

vladmihalcea.com/optimistic-vs-pessimistic-locking/

OS OUTROS MATERIAIS DESTE CONCEITO

#2 ARTIGO EM INGLÊS ~3.000 palavras **Handling Concurrency Without Locks — Haki Benita (2022)**

Cobre a terceira via que o #1 não cobre: **deixar o banco decidir** — `UPDATE ... WHERE saldo >= :v, IntegrityError/EAFP, F()` expressions, RETURNING — em vez de lock explícito na aplicação. É literalmente a tensão que o planejamento do desafio já registrou (“a arquitetura ensinada e a solução correta se contradizem”).

hakibenita.com/django-concurrency

Sem 3º item. As opções em PT-BR encontradas eram específicas de SQL Server (TempDB /Version Store) ou de qualidade de fórum — nenhuma passou na barra de acessibilidade para o piso declarado.

13 Idempotência

#1 BLOG OFICIAL EM INGLÊS ~1.500–2.000 palavras **Designing robust and predictable APIs with idempotency — Stripe (2017)**

Escolha default. É o texto canônico do tema — quem criou a convenção do header Idempotency-Key que virou padrão de mercado (hoje rascunho de padrão IETF). Explica o problema (rede não confiável) antes da solução.

stripe.com/blog/idempotency

OS OUTROS MATERIAIS DESTE CONCEITO

#2 ARTIGO EM PORTUGUÊS ~1.800–2.000 palavras **Idempotência em requisições HTTP — Juan Lira, DIO**

Cobre o mesmo conceito em português, com a tabela de quais métodos HTTP já são idempotentes por definição (GET, PUT, DELETE) e quais não são (POST, PATCH) — informação que o #1 assume conhecida.

www.dio.me/articles/idempotencia-em-requisicoes-http

#3 ARTIGO TÉCNICO EM INGLÊS ~4.000–5.000 palavras **Implementing Stripe-like Idempotency Keys in Postgres — Brandur (2017)**

Mais avançado: mostra a implementação real em Postgres (schema, fases atômicas, máquina de estados de recuperação). Não é piso — é para quem quiser implementar de verdade, não só entender o conceito.

brandur.org/idempotency-keys

14 Erro como caminho previsto do fluxo

#1 ARTIGO EM INGLÊS ~2.500 palavras **Designing API Errors — HTTPToolkit (2024)**

Escolha default. Argumenta exatamente o ponto da R3 do desafio: um status HTTP sozinho não basta, é preciso um type/código específico por falha para que o cliente possa agir por programa (`if (error.type === 'missing_api_version')`), não só exibir uma mensagem.

httptoolkit.com/blog/designing-api-errors/

OS OUTROS MATERIAIS DESTE CONCEITO

#2 ARTIGO EM INGLÊS ~2.000–2.500 palavras **Problem Details (RFC 9457): Doing API Errors Well — Swagger/SmartBear (2024)**

Mostra o formato padronizado (type, title, status, detail, instance) que institucionaliza a ideia do #1 — útil como “olha, isso já virou padrão de mercado (RFC), não é frescura”.

swagger.io/blog/problem-details-rfc9457-doing-api-errors-well/

#3 DOC OFICIAL EM PORTUGUÊS **Referência de Status HTTP — MDN Web Docs (reaproveitado do item 1+2, ângulo diferente aqui)**

É a matéria-prima que os dois primeiros ensinam a organizar: sem entender a semântica correta de cada faixa 4xx/5xx primeiro, o catálogo de erros próprio vira arbitrário.

developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Status

15 RFC / design doc: o que é, como se escreve, por que existe

#1 ARTIGO EM INGLÊS Design Docs at Google — Philip Zeyliger (2020)

TAMANHO 10–20 páginas para projetos grandes, mas aceita "mini design doc" de 1–3 páginas

Escolha default. A referência mais citada do mercado sobre o tema; explica estrutura (contexto, objetivos, design, alternativas consideradas) e, principalmente, **por que existe**: pegar problema de design cedo, quando ainda é barato corrigir. Dá a régua exata que a R7 do desafio pede — "as decisões tomadas e as descartadas".

www.industrialempathy.com/posts/design-docs-at-google/

OS OUTROS MATERIAIS DESTE CONCEITO

#2 ARTIGO EM PORTUGUÊS ~3 min Tomando decisões técnicas usando RFCs — Elton Minetto (2021)

Versão enxuta em português, com processo prático (repositório + PR + merge/close) usado de verdade em times brasileiros — mais fácil de copiar no primeiro RFC do aluno que o modelo do Google.

eltonminetto.dev/post/2021-05-15-rfc/

#3 POST ORIGINAL EM INGLÊS 1–2 páginas por decisão Documenting Architecture Decisions — Michael Nygard (2011)

O texto que criou o formato ADR (contexto → decisão → consequências), hoje prática consolidada (Thoughtworks Radar, categoria "Adopt"). Ângulo complementar ao #1/#2: em vez de "documento antes de construir", é "registro pequeno por decisão", útil quando o RFC completo é exagero.

cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Caminho mínimo — 3 horas antes de 19/09

Ordenado para tocar as restrições que a rubrica cobra (R4 a R8) e fechar nos dois extremos — vocabulário de contrato no início, "como registrar a própria decisão" no fim.

- | | |
|---|---|
| 1 MDN — Visão geral do HTTP
15 min | 2 MDN — Métodos de Requisição HTTP
8 min |
| 3 MDN — Referência de Status HTTP (ler só as faixas 4xx e a nota do 404/403)
8 min | 4 freeCodeCamp — What is REST?
8 min |
| 5 freeCodeCamp PT — Bancos de dados ACID
10 min | 6 rponte.com.br — UUID como PK (R5)
15 min |
| 7 Cultured Systems — soft delete anti-pattern (R4)
12 min | 8 Programadores Deprê — FLOAT pra dinheiro (R6)
10 min |
| 9 BugHunt — IDOR (R8)
12 min | 10 Refactoring Guru — State
12 min |
| 11 DIO — Idempotência em requisições HTTP
10 min | 12 HTTPToolkit — Designing API Errors (R3)
12 min |
| 13 Elton Minetto — RFCs (R7)
5 min | |

Total: ~2h17

Sobra buffer real dentro das 3h para reler o que não fixou de primeira ou para abrir o GitHub Docs de paginação (item 4-1), que fica de fora do caminho mínimo por não ser cobrado diretamente por nenhuma das 8 restrições — mas é o material **mais "mão na massa" da lista inteira.**